



Advice Tracker: how the plug-in works

Design document, version 0.3, for computer-science
readers (no AI knowledge needed)

Wang Bill Zhu
University of California, Berkeley

What the plug-in is for

People ask AI chat assistants for advice about their own lives. The study asks: what happens afterwards? A Chrome extension watches five chat websites and has three jobs:



1 Notice

Find the parts of a chat where the participant asks the AI for advice about their own situation (an advice episode).



2 Ask

When the episode ends, a small panel on the chat page asks “Did you ask for advice here?”, then for three ratings.



3 Follow up

Fourteen days later: what did you decide, did you follow the advice, how did it go?

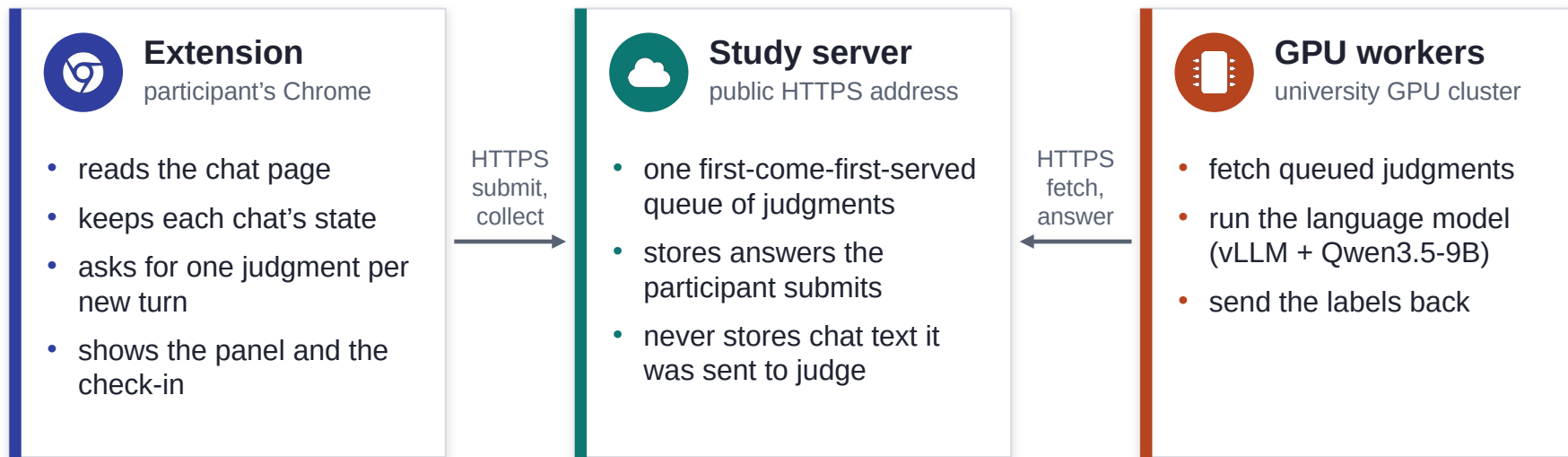
Constraints that shape the design

- **Privacy first:** the research ethics protocol (IRB) limits what may leave the participant’s computer.
- **Our own AI model:** it runs on university GPUs; no commercial AI service sees any text.
- **No inbound connections:** the GPU cluster may open connections, but never accept them.
- **Scale:** 5,000 participants means about 44,000 judgments a day, 92 a minute at peak.

Words used in this document

Word	Meaning
Turn	One message the participant sends, with the AI's reply to it. Turn 3 = the third user message.
Advice episode	Consecutive turns in which the participant asks for and discusses advice about one situation of their own. It has a start, an opening turn (the first request) and an end.
Segment	The turns since the last episode or topic change: what the judge may read as background.
Language model	A program trained on huge amounts of text to continue a text: given instructions and a chat, it writes an answer. We run Qwen3.5-9B, an openly available model, on our own GPUs.
Judge	The language model used as a classifier: instructions (the prompt) and turns go in, a JSON answer with yes/no labels comes out. It can be wrong, so participants confirm.
Label	The yes/no answer of one judgment.
Snapshot	The list of messages shown on the chat page at one moment, as {role, content} pairs.

Three parts, and who connects to whom



Why the workers pull: the cluster forbids inbound connections, so nothing can call the GPUs. Workers ask the server for work over ordinary outbound HTTPS, like a browser would. The participant's browser only ever talks to the server.

A Chrome extension in one page (Manifest V3)



Content script

Runs inside the chat page: reads the page's HTML and draws the rating panel. It keeps no lasting state.



Background worker

A service worker with no page. It wakes up for events; Chrome stops it after about 30 s without one.



chrome.storage.local

A key-value store on disk, private to the extension. It survives the worker stopping and browser restarts.



chrome.alarms

Timers kept by Chrome (here: 1 minute or more). When one fires, Chrome restarts the worker if needed.



Messages

chrome.runtime.sendMessage carries JSON between the content script, the worker and the extension's pages.



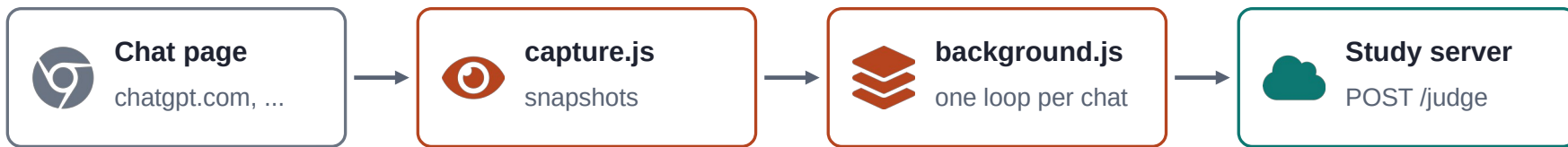
Extension pages

The extension's own HTML pages: the toolbar popup, the settings page and the check-in page.



Design rule: the worker may vanish between any two events, so every step is saved to storage before the next one, and an alarm can always resume it.

Inside the extension: files and data flow



File	What it does
lib/sites.js	per site: CSS selectors for the messages, and where the URL keeps the chat's id
content/capture.js	takes snapshots; notices which user turns the participant sent on this page
lib/episodes.js	the episode state machine, as pure functions (no Chrome calls), tested in Node
background.js	storage, the judging loop, episode records, panel or notification, uploads
content/rating_panel.js	the panel on the chat page, in a shadow DOM so the site's CSS cannot touch it
ui/checkin.*	pending ratings and the day-14 check-ins, in an extension tab
lib/questions.js	every question, used by both the panel and the check-in page
lib/redact.js	removes e-mails, phone numbers, URLs, long numbers, addresses, listed names
popup/, options/	counts and pause; participant ID, sharing default, excluded sites, export, delete all

Reading the chat off the page

1

Find the messages

Each site has CSS selectors for user and AI messages (lib/sites.js); the content script turns them into [{role, content}, ...].

2

Wait for the page to settle

A MutationObserver sees every change. A snapshot is taken once the page has been still for 2 s, and at least every 10 s while a reply is still appearing.

3

Name the chat

The key is the site plus the chat's id from the URL (chatgpt.com/c/<id>). A new chat has no id until its first message: it gets a temporary key, and later an alias.

4

Mark sent turns

A user turn that appears soon after Enter or a Send button was pressed is “sent”. Turns that appear without one (an old chat's history) are never judged.

One snapshot, as capture.js sends it

```
{
  "key": "chatgpt.com/68f1c2...",
  "aliasOf": "chatgpt.com/new-mg3k1",
  "site": "ChatGPT",
  "messages": [
    {"role": "user",
     "content": "I have two offers ..."},
    {"role": "assistant",
     "content": "It depends on what ..."}
  ],
  "sent": ["9f1c2a7b:1"]
}
```

“sent” lists turn ids. A turn id is a hash of the turn's text plus its occurrence number (“9f1c2a7b:1”: the first turn with this text), so a turn can be recognized without keeping its text.

What an advice episode is: the running example

Turn	The participant writes	Turn	The participant writes
1	Summarize this article on remote work.	6	Add a dry-run option.
2	Remote offer, 15% less pay. Should I take it?	7	My sister may stay with us. How do I raise it?
3	My commute is 50 minutes; I have a 2-year-old.	8	She'd pay some rent, if it matters.
4	What should I ask for when I negotiate?	9	Thanks, that really helps.
5	Unrelated: a script to rename my photos.		

episode 1: turns 1–4, opening at 2



episode 2: turns 5–9, opening at 7



Turn 5 changes the subject, so episode 1 ends at turn 4. Episode 2 starts at turn 5: turns 5 and 6 are not advice but stay in as background; the advice request opens at turn 7.

The two yes/no questions asked of the judge



The advice question

Is the participant asking for advice about their own situation?

Sees: the new turn and the segment's earlier user turns (at most 10), without the AI's replies

Stage 1: does it ask for advice or a recommendation? Stage 2: is it about their own decision, not a hypothetical or a task? Advice = yes to both.

```
{"stage1": "yes", "stage2": "yes",  
  "reasoning": "..."} 
```



The continuation question

Does the new turn continue the same thread?

Sees: the previous user turn, the AI's reply to it, and the new turn

Yes for details, follow-up questions or thanks about the same situation; no for a change of subject.

```
{"continuation": "no",  
  "reasoning": "..."} 
```



How good is the judge? These prompts (server/prompts/) were scored on 50 chats that people labeled by hand: of the chats the judge flags, about 60% really ask for advice (precision 0.60), and it finds 94% of those that do (recall 0.94). A missed request cannot be fixed later; a false alarm is removed by the participant's "No".

The state machine of one chat



- **After a switch**, the same turn is asked the advice question at once: a new topic can itself be an advice request. The new segment starts at that turn.
- **An episode also ends** after 10 minutes without activity, when its tab closes, when a turn appears that was not sent here (history), or when an earlier turn is edited.
- **The state is small:** judgedThrough, segmentStart, state, opening, end, the ids of judged turns, and the judgment in flight. No text.

The running example, judgment by judgment

Turn	State before	Question	Answer	What happens
1	searching	advice	no	nothing (the segment starts at turn 1)
2	searching	advice	yes	episode opens at 2 (turn 1 stays as background)
3	episode	continuation	yes	episode now ends at 3
4	episode	continuation	yes	episode now ends at 4
5	episode	continuation	no	episode 1 = turns 1–4 closes (panel); segment from 5
5	searching	advice	no	the switching turn itself is not advice
6	searching	advice	no	nothing
7	searching	advice	yes	episode opens at 7 (turns 5–6 as background)
8, 9	episode	continuation	yes	episode now ends at 9
—	episode	none		10 quiet minutes: episode 2 = turns 5–9 closes (panel)

Ten judgments for nine turns: one per turn, plus one more for the topic switch. Shaded answers: an episode opens (coral), continues (teal), or ends at a switch (gray).

Keeping the state in line with the page



History is never judged

An old chat's turns were not sent here, so they are skipped. Turns typed on another device are skipped too, and end an open episode.



Older turns load above

Some sites load long chats in pieces. When the known turns reappear further down, every stored turn number shifts by the same amount.



An earlier turn is edited

The chat continues from the edited turn k : judgments from k on are forgotten; an episode that began before k closes at turn $k - 1$.



A new chat gets its id

chatgpt.com/ becomes chatgpt.com/c/<id> after the first message: the new key becomes an alias of the temporary one.



Recognizing a turn without its text: its id is a 32-bit FNV-1a hash of the text plus its occurrence number (3c07d5e2:2 is the second “ok”). The state keeps the ids of judged turns; comparing them with the page's ids reveals all four cases.

The background worker's loop, for one chat

```
settle(chat):  
    loop:  
        load the snapshot and the state  
        sync(state, page)    # history, edits  
        if a judgment is in flight:  
            ask the server for its label  
            (the server waits up to 20 s)  
            if done: apply the label  
        else if a turn is not judged yet:  
            submit it; save the job id  
        else:                # all judged  
            if in an episode:  
                if quiet 10 min or tab closed:  
                    close it: record it, ask  
                else: set a quiet alarm; stop  
            delete the snapshot; stop
```

- **One chat, one turn at a time:** the next question depends on the last answer, so a chat's turns are judged in order. Different chats run side by side.
- **Every step is saved first:** the job id goes to storage before the worker waits. An alarm one minute ahead restarts the loop if Chrome stops the background worker meanwhile.
- **Doing a step twice is harmless:** an episode's id is built from its chat, start and opening, so recording it again keeps one copy.
- **Text is kept only while needed:** once every turn is judged and no episode is open, the snapshot is deleted.
- **Asking:** the panel on the chat page, or a notification when the tab is gone.

What the participant sees



When an episode ends

- A panel slides in on the chat page, quoting the turn the judge flagged.
- “Did you ask for advice here?” “No” deletes it and counts as a judge error.
- If the tab is closed: a notification, with no chat text in it.



After “Yes”

- 3 required ratings, 1 to 5: helpful, trust, potentially harmful.
- Optional: the decision in a sentence, the area of life, did the AI recommend, will you act, had you decided.
- Share: answers and the redacted chat (editable), answers only, or nothing.

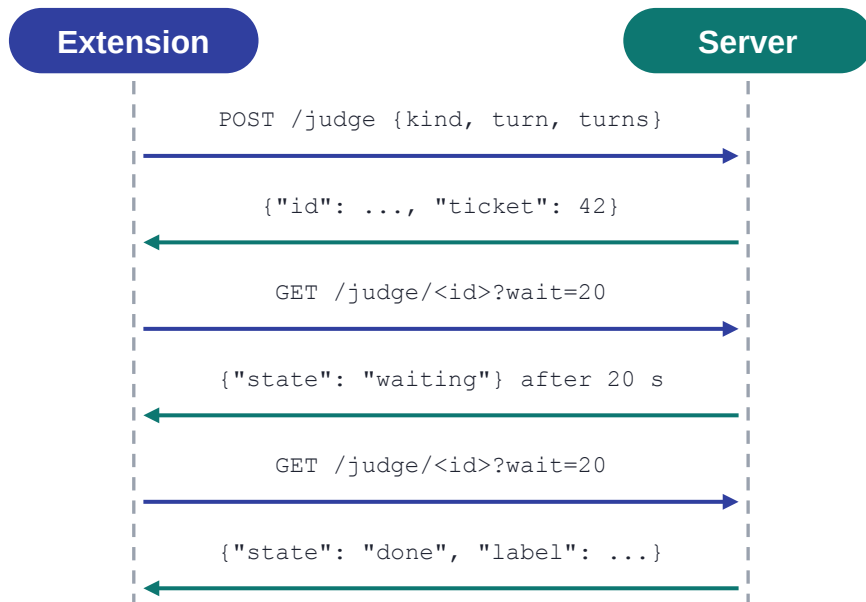


Day 14

- A notification opens the check-in page.
- Outcome: did you follow it, and what did you do?
- Feelings, the AI’s role, who else you asked.
- Looking back: the three ratings again.

Always possible: later (1 hour), skip (deletes the episode), or open the questions in a tab. A number on the toolbar icon counts what is waiting.

Talking to the server: submit, then collect

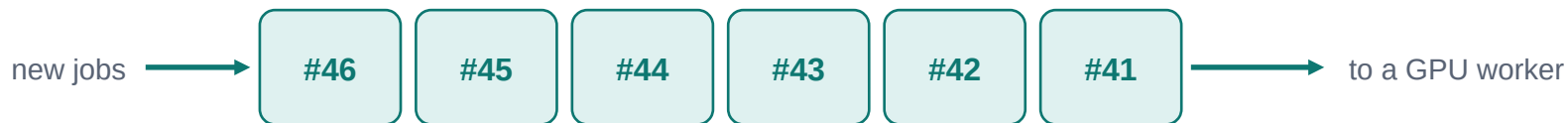


Why two requests: a judgment can take minutes, but Chrome stops a background worker that waits over 30 s for one response. Each wait lasts at most 20 s.

Endpoint	Who	Purpose
<code>POST /judge</code>	extension	queue a judgment
<code>GET /judge/<id></code>	extension	collect the label
<code>POST /events</code>	extension	answers, heartbeat
<code>GET /health</code>	anyone	queue length
<code>GET /worker/next</code>	GPU worker	take the next job
<code>POST /worker/result</code>	GPU worker	labels or error

Extensions send a secret study key with every request, GPU workers a separate worker key; only `/health` is open. The server holds each job's texts in memory only until it is judged.

One queue, first come, first served



Each judgment gets a ticket number when it arrives. Workers always take the oldest job, so nobody's turn overtakes an earlier one.

Rule	Why
A job a worker took but did not answer in 600 s fails	the worker died; the extension submits again 2 minutes later
A queued job nobody asked about for 600 s is dropped	the browser was closed; no point judging it
A label nobody collected in 600 s is forgotten	the server keeps no labels it does not need
Over 2,000 waiting jobs: the server answers 503 (busy)	protects the server; the extension retries
Texts live in memory only, until the judgment ends	the log file keeps labels and timings, never text

GPU workers pull their work



One GPU job on the cluster

```
64 of these loops run at once:  
job = GET /worker/next      (waits <= 20 s)  
label = judge(job)  
POST /worker/result {id, label}
```

vLLM, a program that serves the model on the same node, batches many judgments into one pass over the GPU, which is where the speed comes from.

- **Pull, not push:** the GPU worker opens every connection, so the cluster accepts none.
- **Always running:** cluster jobs have time limits, so each job queues its own successor when it starts.
- **A live backup:** a second job pulls from the same queue; each job goes to exactly one worker.
- **Sizing:** 5,000 participants need about 92 judgments a minute at peak: 3 A100 or 2 H100 GPUs, plus a spare.



The model: Qwen3.5-9B with 8-bit weights and “thinking” on: it writes out its reasoning before the answer, which makes it more accurate but slower (up to 12,288 tokens per judgment; a token is roughly a word).

Where data lives, and for how long

Where	What	How long
Browser: snap:<chat>	the chat's messages (the snapshot)	while a turn is unjudged or an episode is open
Browser: chat:<chat>	turn numbers and turn ids; no text	the study
Browser: episode:<id>	the episode's messages; after answering, only the answers and the first request	messages: until answered or skipped; the request: until day 14
Server memory	texts of judgments in the queue	until judged (seconds to minutes)
Server: judgments.jsonl	labels, timings, participant ID; no text	the study
Server: events.jsonl	answers; the redacted chat if shared	the study, then archived
GPU worker	nothing on disk; its log has job ids and labels	—

Chats that turn out not to be advice keep their text only until their turns are judged, usually under a minute.

Privacy rules, and the code that enforces each

Rule	Enforced in
Only the five chat sites are read	manifest.json (host permissions)
Only turns sent while the extension runs are judged	capture.js (send detection), Episodes.sync
The AI's reply is sent only to judge continuation	Episodes.nextJob
The server never stores the texts it judges	app.py (finish drops the texts, logs labels)
Nothing the participant answers leaves before Submit	background.js (uploads only on Submit)
Shared chats are redacted and editable before sending	redact.js, the panel's transcript box
Not advice or skipped: deleted at once	background.js (onNotAdvice, onDismissed)
Notifications never show chat text	background.js (askToRate, the check-in alarm)
The heartbeat carries counts only	background.js (heartbeat)
Pause, exclude a site, export, delete all	the popup and the settings page

When things go wrong

Failure	What happens
Participant offline	submitting fails: retried every 2 minutes; answers wait in a local queue (30-minute retries)
Server restarts	queue lost; collecting returns 404, so the turn is submitted again
No GPU worker running	jobs wait in the queue; extensions keep asking; jobs of closed browsers are dropped
A GPU worker dies mid-job	after 600 s the job fails; the extension submits it again
Chrome stops the background worker	the job id is in storage; the next alarm resumes collecting (tested)
Browser restarted	on startup, every chat that still has text is resumed
Unreadable judge answer	the GPU worker asks the model again, up to 3 times, then reports a failure
Site HTML changes	nothing is captured; the popup's Show captured messages button reveals it; fix lib/sites.js

How it is tested



Unit tests

- Judge prompts and their rendering are byte-identical to the evaluated pipeline (on 300 generated chats).
- Server: queue order with 6 clients, timeouts, the real worker code, no text in logs.
- The state machine finds the same episodes as the Python reference on 500 random chats.



Browser end-to-end

- Real headless Chromium, the extension, a mock chat page, and the real server with a judge that follows fixed rules.
- New chat, switch, quiet, Yes and No, reload, past chat, closed tab, check-in.
- The background worker is stopped in the middle of a 70 s judgment.



Smoke test

- Replays four chats against a live server and the real model, turn by turn.
- Uses the extension's own lib/episodes.js.
- The running example should give episodes 1–4 and 5–9.

```
python tests/test_judge.py; python tests/test_server.py; node tests/test_episodes.js  
srn --jobid=<job> --overlap bash tests/run_e2e.sh; STUDY_KEY=... node tests/smoke_test.js <url>
```

Known limits and next steps



Known limits

- **Site HTML changes:** selectors break without warning.
- **Send detection:** a turn sent by clicking a suggested prompt or by voice counts as history.
- **One server process:** the queue lives in memory; a restart loses it (extensions resubmit).
- **Judge errors:** about 4 in 10 chats the judge flags contain no real advice request; participants' "No" removes them.
- **Not covered:** desktop and mobile apps.



Before the full study

- **Check every site** with the popup's "Show captured messages" button, before launch and during the study.
- **Host the server** where sensitive research data may be stored (UC protection level P3), with a persistent queue.
- **Load test** with simulated extensions at the expected peak.
- **Rotate the keys** of the hosted server, and keep a spare GPU worker running.